

Spark vs. Ray: The Data Engineering Duel

DA3408 Assignment 3

Rohan Sai P. (DA24B049)

October 2026

1 Overview

On the same 4-CPU, 4 GB cluster, Ray completed the full pipeline in 141.6 s, compared with 281.5 s for Spark. Ray required memory-specific configuration to complete and spilled substantially more data to disk. Both frameworks produced identical output (18,717,045 rows, matching row-level checksums).

Objective. Implement one data preprocessing pipeline in Apache Spark and in Ray Data, run each on a two-worker cluster, and compare execution time, resource use and Python UDF overhead.

Dataset. NYC TLC Yellow Taxi trips, January to June 2023: six Parquet files (305 MB compressed, 2.0 GiB when converted to CSV with all 19 columns; 18.7 million rows survive cleaning), and the 265-row taxi zone lookup CSV.

Pipeline (identical in both frameworks).

1. Ingestion: read all six monthly files from a shared directory, keep 8 columns, with one fixed schema.
2. Cleansing: drop rows with nulls, drop duplicates, cast timestamps, keep trips with distance > 0 and dropoff after pickup.
3. Transformation: join each trip to its pickup borough and zone; compute per-trip speed (mph) with a custom Python UDF; drop trips above 100 mph; compute average speed per pickup hour and attach it to every trip.
4. Export: write the result to Parquet for ML training.

Environment. MacBook (16 GB RAM) running an Ubuntu 26.04 arm64 VM in UTM with 4 cores and 8 GB RAM. Both clusters run in Docker Compose: one coordinator plus two workers with 2 CPUs and about 2 GB each. Spark 3.3.4 (Java 17, Python 3.10), Ray 2.59.0 (Python 3.11). Each cluster was benchmarked alone, with the other shut down.

2 Cluster Setup

Both clusters use one coordinator and two workers in Docker Compose, with the same compute and memory budget per worker.

	Spark 3.3.4	Ray 2.59.0
Image	eclipse-temurin:17-jdk-jammy + Python 3.10 + pyspark 3.3.4	python:3.11-slim + ray[default,data]
Coordinator	spark-master, port 7077; runs the driver	ray-head, GCS on port 6379; runs the driver, -num-cpus=0
Workers	spark-worker-a, spark-worker-b	ray-worker-a, ray-worker-b
CPU per worker	SPARK_WORKER_CORES=2	-num-cpus=2
Memory per worker	SPARK_WORKER_MEMORY=2G, executor heap 1.5 GB	-memory 0.5 GiB + -object-store-memory 1.5 GB
Shuffle partitions	spark.sql.shuffle.partitions=8	num_partitions=8 on each groupby
UI	Master UI 9090, application UI 4040	Dashboard 8265
Shared data	./data mounted at /opt/spark-data	same folder mounted at /opt/data

Table 1: Cluster configuration.

Networking. Docker Compose places each cluster's containers on a private bridge network, so workers reach the coordinator by service name (spark://spark-master:7077, ray-head:6379). Only the UI ports are published to the VM.

Ray-specific settings. shm_size: 2gb on every container, because Ray's object store lives in shared memory and Docker's default is 64 MB. -dashboard-host=0.0.0.0 exposes the dashboard outside the container. -block keeps ray

start in the foreground so the container stays alive. `restart: on-failure` lets a worker retry if it starts before the head is ready.

Fair budgets. For both frameworks, these settings are scheduling budgets rather than hard limits.

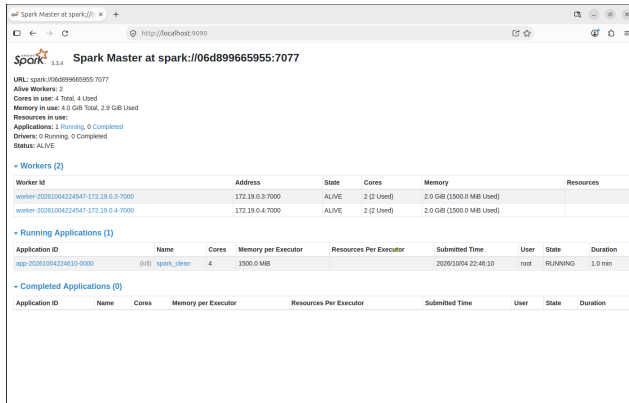


Figure 1: Spark Master UI (port 9090) during a run: 2 alive workers with 2 cores and 2.0GiB each, all 4 cores used by `spark_clean`.

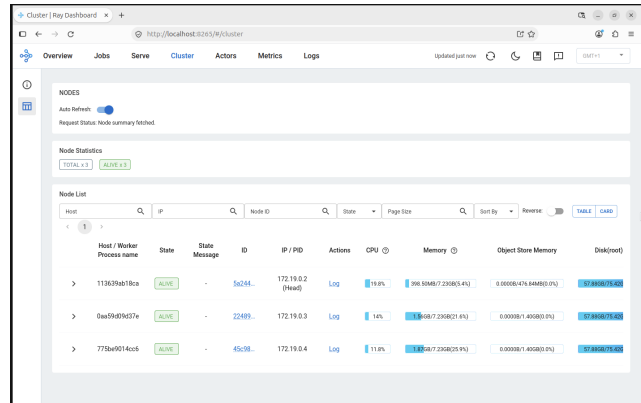


Figure 2: Ray Dashboard cluster resource view (port 8265): head node and two workers ALIVE, with 1.40 GB object store per worker.

3 Pipeline Implementation and Parity

The two outputs are identical: 18,717,045 rows each, with matching row-level hash checksums.

Step	Spark (<code>spark_clean.py</code>)	Ray (<code>ray_clean.py</code>)
Read + fixed schema	<code>spark.read.parquet</code> per file, cast to SCHEMA, <code>unionByName</code>	<code>read_parquet</code> per file, <code>astype(SCHEMA)</code> in <code>map_batches</code> , <code>union</code>
Nulls, filters	<code>dropna</code> , <code>filter</code>	pandas <code>dropna</code> and boolean masks in <code>map_batches</code>
Duplicates	<code>dropDuplicates()</code>	hash each row into 8 buckets, <code>groupby("bucket").map_groups(drop_duplicates)</code>
Zone join	broadcast join (Spark chose it for the 265-row table)	zone table used by every task, pandas <code>merge</code>
Speed UDF	<code>F.udf(speed_mph)</code> , row at a time	same <code>speed_mph</code> function in a Python loop over each batch
Hourly average	<code>groupBy("pickup_hour").agg(avg)</code> , joined back	<code>groupBy("pickup_hour").mean()</code> , merged back
Reuse	<code>.cache() + count()</code>	Parquet checkpoint, streamed back
Export	<code>write.parquet</code>	<code>write_parquet</code>

Table 2: Equivalent operations in the two pipelines.

Schema drift. `passenger_count` is stored as double in January and as `int64` from February onward. Reading the folder in one call made Spark fail on February. Both scripts now read each file separately and cast every column to one fixed schema before combining.

Parity check. `parity.py` reads both outputs in batches, normalises types (timestamps to microseconds, IDs to `int64`), hashes every row and sums the hashes. The sum ignores row order but changes if any value in any row differs. Both outputs gave 18,717,045 rows and the fingerprint 10158552855737734196. The 24 hourly average speeds agree to within 7.1×10^{-14} mph.

The derived feature varies as expected across the day: average speed is approximately 20.8 mph at 5 AM and 9.8 mph at 3 PM.

4 Performance Analysis

Ray completed the full pipeline in 141.6 s; Spark took 281.5 s.

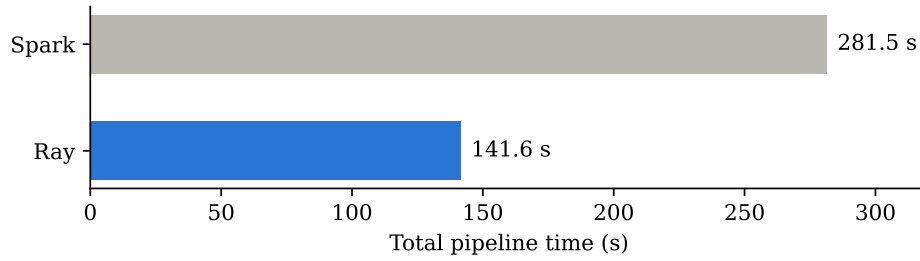


Figure 3: Total pipeline time, ingestion to export, on six months of data. Ray completed the pipeline in about half the time of Spark.

Each framework was measured in one full run, the run shown in the screenshots below. Both runs were recorded with a browser open on the monitoring UIs.

Resource use was recorded with `monitor.sh`, which logs `docker stats` per container every 1 to 2 s, and summarised with `peaks.py`.

Metric	Spark	Ray
Total pipeline time	281.5 s	141.6 s
Cluster peak CPU (400% = 4 cores)	368%	372%
Cluster peak memory	4,535 MiB	5,907 MiB
Worker peak CPU (a / b)	265% / 270%	191% / 188%
Worker peak memory (a / b)	2,046 / 2,066 MiB	2,937 / 3,023 MiB
Coordinator peak CPU	162%	131%
Coordinator peak memory	655 MiB	1,296 MiB
Spilled to disk	806 MiB of cached table (Fig. 9)	several GB per worker; object stores at about 90% during the run (Fig. 10)

Table 3: Execution time and peak resource utilisation.

Both frameworks used nearly all 4 cores at their peak. Ray’s cluster used 30% more memory than Spark’s (5,907 against 4,535 MiB).

Why Spark’s workers exceed 200%. `SPARK_WORKER_CORES=2` limits concurrent tasks, but the JVM adds garbage collection and shuffle threads, and each task starts a separate Python process for the UDF. Ray’s tasks are the Python processes themselves, so its workers stay near their 2-CPU allowance.

Why Ray spills more. Ray Data holds intermediate blocks in each node’s object store (1.5 GB per worker). The deduplication shuffle must hold the entire cleaned dataset at once, so both object stores ran at about 90% capacity and Ray spilled several GB of blocks to disk per worker, against 806 MiB for Spark’s cache. Spark retained only the explicitly cached table and streamed all other data through each stage. Despite the heavier disk traffic, Ray completed sooner, as its batches are processed by pandas directly in Python worker processes rather than through a JVM-to-Python boundary.

Ray memory configuration. Two choices, within the same 2 GB per-worker budget as Spark, allow Ray to complete this workload:

1. Each worker’s budget is weighted toward data: `-memory 0.5 GiB` for task scheduling and `-object-store-memory 1.5 GB` for data blocks.
2. The feature table is checkpointed to Parquet rather than held with `.materialize()`. The 18.7-million-row table is written to disk once in compressed form and streamed back for the hourly average and the final merge.

Screenshots of both runs (console output, Spark UI and Ray Dashboard) are in Appendix A (Figures 5 to 10).

5 UDF Deep-Dive

The JVM boundary makes Spark’s Python UDF $21\times$ slower than the same formula as a native expression; Ray removes that boundary, but a row-by-row Python loop is still slow in either framework.

Method. After export, each script reads its own output back, caches it, and times the same `speed_mph` function two ways: as a row-by-row Python function, and as a fast path (a native Spark column expression, or a vectorized pandas expression in Ray). Each result is summed so the work cannot be skipped.

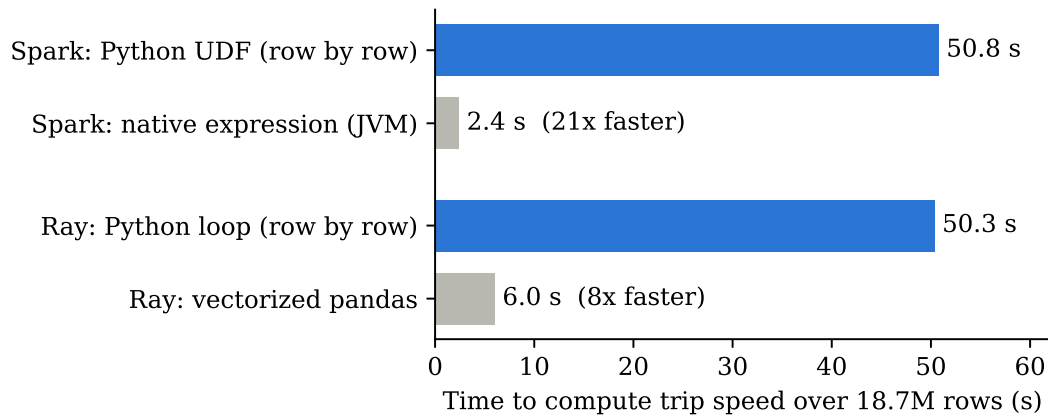


Figure 4: Time to compute trip speed over 18.7 million rows, one run each. Ray times include reading data blocks back from disk after spilling.

Spark: JVM serialization overhead. Spark’s engine runs in Java. For a Python UDF, each executor serializes rows, sends them to a separate Python worker process, runs the function and sends results back. The native expression is compiled to bytecode. The gap is 50.8 s against 2.4 s.

Ray: native Python execution. Ray tasks are Python processes, so no Java boundary is crossed: the function executes on the node where the data block is present. Even so, Ray’s loop took 50.3 s, essentially the same as Spark’s 50.8 s, for two reasons. Part of the benchmark input was read back from disk after spilling, and looping over a pandas column passes pandas `Timestamp` objects, whose arithmetic is slower than the plain `datetime` objects Spark hands its UDF.

Interpretation. Ray’s Python-native advantage is that whole Arrow/pandas batches flow into ordinary Python code, so vectorized NumPy or pandas logic runs at full speed.

6 Winner and Recommendation

Fastest for this task: Ray. Ray completed the pipeline in 141.6 s against Spark’s 281.5 s, about twice as fast, with identical output. This result depends on configuration: Ray required a hash-bucket deduplication, per-batch sums instead of an aggregator, the sort-based shuffle and a Parquet checkpoint to complete within the 2 GB per-worker budget, and it spilled several GB to disk per worker. Spark completed the same workload with its default operators and spilled 806 MiB. Spark is therefore the more robust choice for this workload, and Ray the faster one once configured.

	BI-first project	AI-first project
Typical work	SQL-style cleaning, joins, aggregates, reports	Feature engineering in Python, model inference, training data loaders
Choice	Spark	Ray
Why	Built-in <code>dropDuplicates</code> , joins and group-bys; optimizer pushes filters and picks broadcast joins; spills gracefully under memory pressure	Native Python execution: pandas and NumPy batches are passed to tasks without a JVM boundary; schedules GPUs and actors; streams data directly into training and inference
Limitations	Python UDFs cross the JVM boundary (21× here); use built-ins or <code>pandas_udf</code>	Needs more memory per node; shuffle-heavy steps spill heavily and hash shuffle can stall on small nodes

Table 4: Framework choice by project type.

7 Performance Tuning Notes and AI Attribution

Every change below was suggested by an AI assistant (Claude, Anthropic) while debugging, then applied and verified on my own runs.

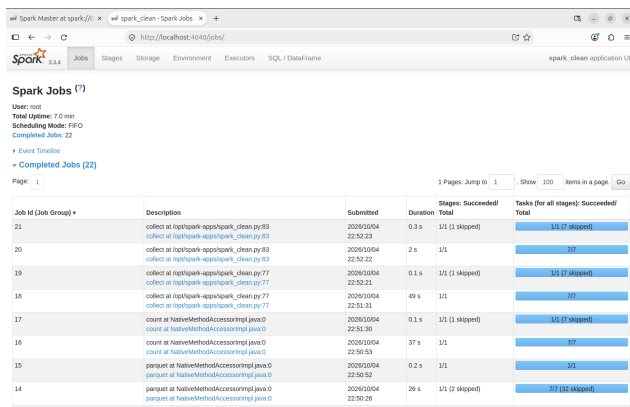


Figure 7: Spark application UI (port 4040), Jobs tab: the UDF benchmark (job 18, 49 s) against the native expression (job 20, 2 s).

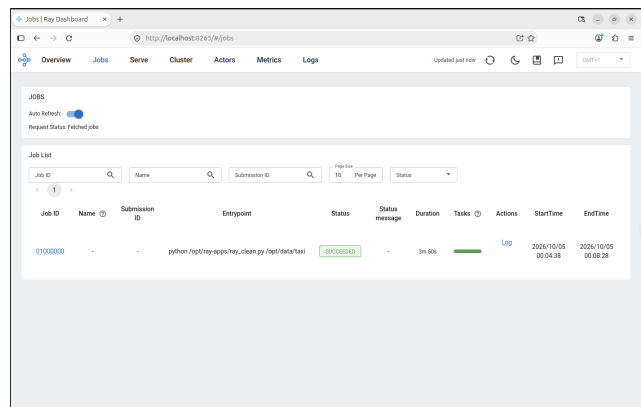


Figure 8: Ray Dashboard Jobs tab: the run SUCCEEDED in 3 min 50 s, including startup and the UDF benchmark.

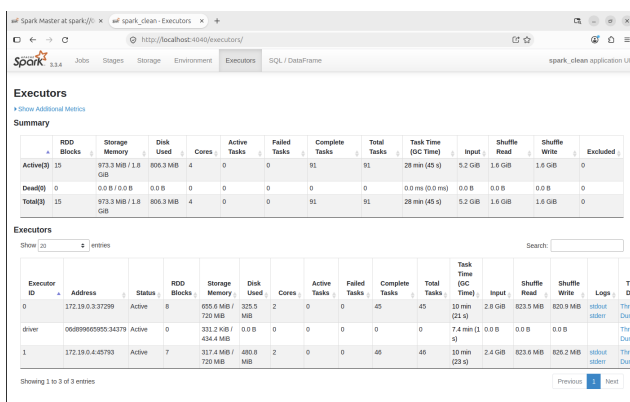


Figure 9: Spark Executors tab: both workers ran an even share of the job (45 and 46 tasks), with 806 MiB of the cached table spilled to disk.

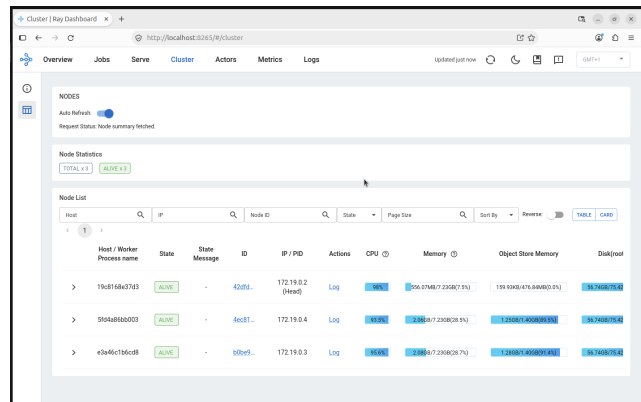


Figure 10: Ray Dashboard during the run: both worker object stores at about 90% of their 1.40 GB capacity.